SISTEMAS DIGITALES

Semana 2

Dando el salto: Del ATmega328P al ESP32

Usamos como ejemplo de comparación una tienda de barrio (el Arduino Uno con su ATmega328P) a un hipermercado completo (el ESP32). Aunque la lógica interna para controlar los pines sigue siendo la misma, las especificaciones cambian:

Característica	ATmega328P (Semana 1)	ESP32-WROOM-32
Ancho de palabra	8 bits	32 bits
Núcleos y Velocidad	1 núcleo a 16MHz	2 núcleos Xtensa LX6 hasta 240MHz
Memoria SRAM	2KB	520KB
Memoria Flash	32KB (interna en el chip)	4MB (chip aparte conectado por SPI en el módulo)
Tensión Lógica	5V	3.3V
Asignación de Pines	Fija (por ejemplo, PB5 es siempre el bit 5 de PORTB)	Configurable gracias a una central llamada *GPIO matrix*

Cómo se controlan los pines en el ESP32

En el ESP32 ya no manejamos los clásicos registros DDRB o PORTB de 8 bits porque ahora trabajamos con arquitecturas de 32 bits. Sin embargo, la idea fundamental es la misma: configurar el sentido, escribir salidas y leer entradas mediante registros mapeados en memoria:

Función del Pin	Equivalente en AVR	Equivalente en ESP32 (GPIO 0-31)
Dirección (Entrada / Salida)	DDRB	GPIO_ENABLE_REG
Escribir la Salida (Alto / Bajo)	PORTB	GPIO_OUT_W1TS_REG / GPIO_OUT_W1TC_REG
Leer la Entrada	PINB	GPIO_IN_REG
Resistencia Pull-up interna	PORTB (configurado como entrada)	Registro IO_MUX independiente de cada pin

Nota: Los pines del 32 al 39 no entran en un solo registro de 32 bits, por lo que utilizan un segundo banco de registros (GPIO_OUT1_REG, GPIO_IN1_REG).

Operaciones atómicas con W1TS y W1TC

Para evitar el riesgo de que una interrupción o el segundo núcleo modifiquen un registro a mitad de camino y se pierda información, el ESP32 utiliza dos registros especiales que escriben en un solo paso sin necesidad de leer antes:

- **W1TS (*Write 1 to Set*):** Los bits en 1 se encienden; los bits en 0 no se tocan.
- **W1TC (*Write 1 to Clear*):** Los bits en 1 se apagan; los bits en 0 no se tocan.

Precauciones con la placa DevKit V1 y Pines de Strapping

- Límites eléctricos (3.3V): Los pines del ESP32 operan a 3.3V y no toleran 5V directos; conectar un sensor de 5V quemará el pin de entrada. Para los LEDs, se recomiendan colores rojo, verde o amarillo con resistencias de 220 – 330 Ω (los LEDs azules y blancos casi no encienden a 3.3V con estas resistencias).
- Pines de Strapping: Son pines especiales que el chip lee justo al arrancar para saber en qué modo ponerse:
  - *GPIO0 (Botón BOOT):* Debe estar en BAJO al encender para entrar en modo de carga de código.
  - *GPIO2:* Debe estar libre o en bajo al arrancar (suele tener el LED azul integrado).
  - *GPIO12:* Si está en ALTO al encender, cambia la tensión de la flash a 1.8Vy la placa no arranca.
  - *Regla de oro:* Evita conectar sensores o botones complejos a los pines de strapping para prevenir dolores de cabeza con el hardware real.

¿Cómo vive y arranca un programa embebido?

- A diferencia de una computadora donde abres un programa (main() termina y el Sistema Operativo recupera el control), un programa embebido es como un guardia nocturno: arranca con la energía, configura todo una vez y se queda dando rondas por siempre en un bucle infinito hasta que se corta la luz.
- Por debajo, todo sketch de Arduino en el ESP32 corre sobre un sistema operativo de tiempo real llamado FreeRTOS (específicamente en la tarea loopTask del núcleo 1).
- El Superlazo: La estructura clásica de Arduino ejecuta primero setup() una sola vez (para inicializar timers, pines y ADC) y luego repite loop() eternamente dentro de un bucle infinito for (;;) { loop(); }.

Tipos de datos en C y el peligro de los desbordamientos

Una variable es simplemente una caja de memoria en la SRAM cuyo tamaño determina qué números puede guardar y cómo se interpretan sus bits. Lo peligroso es que el tamaño de int cambia según la plataforma (en un Arduino Uno de 8 bits mide 2 bytes, mientras que en el ESP32 de 32 bits mide 4 bytes).

Para evitar errores y asegurar portabilidad, usamos los tipos explícitos de la librería <stdint.h>:

Tipo de Dato	Bits / Bytes	Rango de Valores	Uso Recomendado
int8_t / uint8_t	8 bits (1 byte)	-128 a 127 / 0 a 255	Registros, matrices pequeñas, optimización de memoria
int16_t / uint16_t	16 bits (2 bytes)	~ 32.7k / 0 a 65.5k	Contadores medianos
int32_t / uint32_t	32 bits (4 bytes)	~ 2.1 * 10^9 / 0 a 4.29 * 10^9$	Tiempos (millis), acumuladores grandes

- **El Desbordamiento:** Si una variable sin signo (uint8_t) llega a su límite (255) y le sumas 1, da la vuelta automáticamente a 0 (como un odómetro de carro). Si usas variables con signo (int), el desbordamiento produce un comportamiento indefinido y puede colapsar el sistema.
- **Literales y Máscaras:** Al hacer desplazamientos de bits grandes (como 1 << 31), un 1 normal se interpreta como entero con signo y desborda. Siempre debes usar sufijos explícitos como 1UL (*unsigned long*) para evitar errores al compilar.

Tiempo sin bloquear: Del delay(), a la función millis()

- Usar delay() es como poner a cocinar a un chef mirando fijamente el horno sin poder atender nada más: congela y bloquea todo el microcontrolador, haciendo que los botones dejen de responder.
- La alternativa profesional es millis() (que mide los milisegundos transcurridos desde el encendido), aplicando el patrón del reloj de pared:
    1. Leer el tiempo actual (millis()).
    2. Restar: ahora - tAnterior para ver cuánto tiempo ha pasado.
    3. Si ya se cumplió el período deseado, actualizar el registro y ejecutar la acción.
    4. Seguir de largo con el resto del código sin bloquear jamás el bucle.

Pull-up, rebote y flanco

- Resistencia Pull-up: Para evitar que un pin quede "al aire" leyendo ruido eléctrico impredecible, habilitamos la resistencia interna de pull-up del ESP32 (pinMode(PIN_BTN, INPUT_PULLUP)). Con esto, cuando el botón está suelto lee HIGH (1), y al presionarlo se conecta a GND leyendo LOW (0).
- El Rebote Mecánico: Al presionar un botón físico, los resortes metálicos vibran microscópicamente durante unos milisegundos, generando falsos múltiples contactos. Para solucionarlo sin usar delay(), aplicamos un antirrebote por software exigiendo que la señal permanezca estable entre 20 y 50ms (por ejemplo, 30ms)
- Detección por Flanco (y no por nivel): Si programas tu código preguntando si el botón *está presionado* (if (pin == LOW)), el microcontrolador sumará miles de activaciones por segundo mientras mantengas el dedo encima. La forma correcta es detectar el flanco de bajada (exactamente el preciso instante de transición de HIGH a LOW), para que la acción cuente una sola vez por cada pulsación humana.